

Advanced Python Programming Hands-on Activity Worksheet

Duration: 30 Minutes

Instructions:

- Complete this worksheet while performing the activity.
- Type your answers directly in this document.
- Write your Python program in VS Code/IDLE.
- Take ONE screenshot showing both your code and output.
- Paste the screenshot at the end of this document.
- Save this file as PDF and submit it to the instructor.

Student Name: Aditya Deshpande

Roll Number: 01

Division: 9th

Date: 09-07-2026

Part A – Think Before You Code

1. What should be the main class for this program? Why?

Answer: Student, because the program is about managing student information.

2. List at least FOUR attributes that every Student should have.

Attribute 1: Name

Attribute 2: Roll Number

Attribute 3: Division

Attribute 4: Marks

3. List at least THREE methods (actions) that a Student object should perform.

Method 1: show_details()

Method 2: result()

Method 3: update_marks()

Part B – Coding Task

Create a Python program for a simple Student Management System.

Requirements:

- ☐ Create a class named Student.
- ☐ Add at least FOUR attributes.
- ☐ Add at least TWO methods.
- ☐ Create THREE student objects.
- ☐ Call the methods for each object.
- ☐ Display the output neatly.

Part C – Reflection

1. What was the easiest part of today's activity?

- Creating the student objects.

2. What was the most challenging part?

- It wasn't that difficult but took me time to decide which method to use for the class.

3. One new thing I learned today:

- I used my previous knowledge of Class and Objects to create this program.

Submission

Paste your screenshot (Code + Output) below.

Code -

class Student:

```
def __init__(self, name, roll_no, division, marks):
```

```
    self.name = name
```

```
    self.roll_no = roll_no
```

```
    self.division = division
```

```
    self.marks = marks
```

```
def show_details(self):
```

```
print("Name:", self.name)
print("Roll No:", self.roll_no)
print("Division:", self.division)
print("Marks:", self.marks)
```

```
def result(self):
    if self.marks >= 35:
        print("Result: Pass")
    else:
        print("Result: Fail")
    print("-----")
```

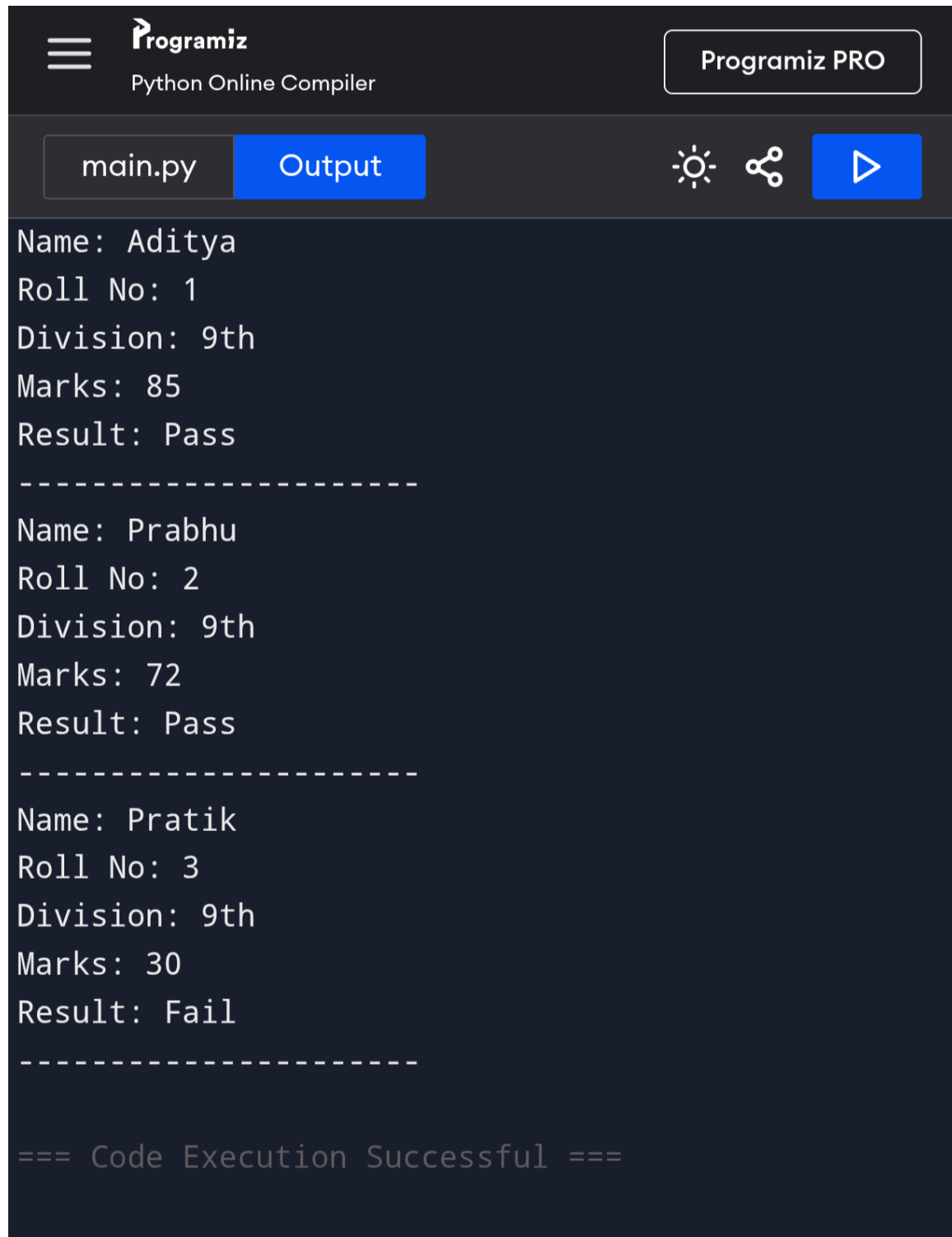
```
s1 = Student("Aditya", 1, "9th", 85)
s2 = Student("Prabhu", 2, "9th", 72)
s3 = Student("Pratik", 3, "9th", 30)
```

```
s1.show_details()
s1.result()
```

```
s2.show_details()
s2.result()
```

```
s3.show_details()
s3.result()
```

Output -



The screenshot shows the Programiz Python Online Compiler interface. At the top, there is a header with the Programiz logo, the text "Python Online Compiler", and a "Programiz PRO" badge. Below the header, there is a navigation bar with a tab labeled "main.py" and a blue button labeled "Output". To the right of the "Output" button are three icons: a sun (theme toggle), a share icon, and a play button (run). The main area displays the output of the script, which consists of three data entries separated by dashed lines. The first entry is for Aditya (Roll No: 1, Division: 9th, Marks: 85, Result: Pass). The second entry is for Prabhu (Roll No: 2, Division: 9th, Marks: 72, Result: Pass). The third entry is for Pratik (Roll No: 3, Division: 9th, Marks: 30, Result: Fail). At the bottom, it says "=== Code Execution Successful ===".

```
Name: Aditya
Roll No: 1
Division: 9th
Marks: 85
Result: Pass
-----
Name: Prabhu
Roll No: 2
Division: 9th
Marks: 72
Result: Pass
-----
Name: Pratik
Roll No: 3
Division: 9th
Marks: 30
Result: Fail
-----

=== Code Execution Successful ===
```